DLDCA Outlab: Structs, Arrays, and Addressing

Week 5

September 1, 2026

Introduction

This outlab is about composition, not new syntax. Every task here builds entirely out of tools you already have – registers, branching, loops, and arrays (including the local-label struct trick from the inlab, `[rect_a.x1]` and friends) – and asks you to combine them more carefully than before. There are no functions and no stack in this outlab; everything lives in `_start` and a handful of registers.

Necessary tools

We shall be using `nasm` to assemble x86-64 assembly into object files, and `ld` to link those object files into an executable. Before starting, make sure both are installed:

```
$ nasm -v
NASM version 2.16.01 compiled on ...
$ ld -v
GNU ld ...
```

Each task can be built and run directly with the provided makefile:

```
$ make task1
...
Return Code: 4
```

As before, every task in this outlab is graded purely by its exit code – you can check this yourself at any time with `echo $?` right after running your program.

Structure of submission/templates

Submission format:

```
your-roll-no/
|- task1/
    |- streak.s (TODO)
|- task2/
    |- histogram.s (TODO)
|- task3/
    |- minesweeper.s (TODO)
```

The folder structure of the expected submission for this outlab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

Task 1: Longest Streak

You are given an array of 64-bit integers, `values`. Find the length of the longest run of *consecutive equal* values.

values	4	4	4	7	7	2	2	2	2	9	9	1	7	7	7

(As with earlier tasks, you should be able to just look at the array above and see the answer: the four consecutive `2`s are the longest run, so the expected answer is `4`. The later pair of `7`s doesn't extend that run. Repeats only count if they're back-to-back.)

Fill in `_start` in `streak.s` and exit with the length of the longest run as the exit code.

To run your code:

```
$ make task1
...
Return Code: 4
```

Task 2: Most Common Letter

You are given a lowercase string, `text`, and a 26-element array, `counts`, already zeroed for you. Build a histogram of letter frequencies into `counts`, then find the *highest* count in the histogram.

With `text = "mississippi"`, both `i` and `s` appear 4 times each.

Fill in `_start` in `histogram.s` and exit with the highest count found.

To run your code:

```
$ make task2
...
Return Code: 4
```

Task 3: Minesweeper

You are given a `ROWS` × `COLS` grid, stored flat in `grid`, where `grid[r*COLS+c]` is `1` if that cell holds a bomb and `0` otherwise. For every non-bomb cell, count how many of its up to 8 neighbors (including diagonals) are bombs.

Your job: count how many non-bomb cells have a neighbor-bomb-count of exactly `0`.

0	0	**1**	0	0	0
0	0	0	0	**1**	0
0	**1**	0	0	0	0
0	0	0	0	0	0
1	0	0	**1**	0	0

(bombs in **bold**)

Some things to note:

- Cells on an edge or in a corner have *fewer* than 8 neighbors. Check that each neighbor's row and column actually lie within the grid before reading it. Reading outside `grid` won't necessarily crash, it'll just silently read whatever memory happens to sit next to it.
- Bomb cells themselves don't get a number in Minesweeper.

Fill in `_start` in `minesweeper.s` and exit with the count of non-bomb, zero-neighboring-bombs cells.

To run your code:

```
$ make task3
...
Return Code: 3
```

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` ← `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` ← `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` ← `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` ← address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` ← `reg` + `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` ← `*mem` + `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` ← `reg` − `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` ← `*mem` − `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` ← `reg/*mem` + 1
`dec`	`reg/*mem`	`reg/*mem` ← `reg/*mem` − 1
`neg`	`reg/*mem`	`reg/*mem` ← −`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` ← ¬`reg/*mem` (One's complement/bitwise)
`imul`	`reg, reg/*mem/imm`	`reg` ← `reg` × `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` ← `*mem` × `reg/imm`
`div`	`reg`	`rax` ← `(rdx:rax)`/`reg`
`div`	`reg`	`rdx` ← `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` ← `reg` ∧ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` ← `*mem` ∧ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` ← `reg` ∨ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` ← `*mem` ∨ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` ← `reg` ⊕ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` ← `*mem` ⊕ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem` - `reg/imm`. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jz`	`label`	Jump if Zero Flag (ZF) = 1
`jne / jnz`	`label`	Jump if Zero Flag (ZF) = 0
`jl`	`label`	Jump if Less (SF ≠ OF)
`jle`	`label`	Jump if Less or Equal (ZF = 1 or SF ≠ OF)
`jg`	`label`	Jump if Greater (ZF = 0 and SF = OF)
`jge`	`label`	Jump if Greater or Equal (SF = OF)
`cmov`*cc*	`reg, reg/*mem`	`reg` ← `reg/*mem` if *cc* holds, else unchanged.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it